

Education Management System

Project Concepts • Advantages • Implementation • Output Checking

■■■■■ guide ■■■■■ current Education Management System project-■ base ■■■■■, project-■■ use ■■■■■■ concepts, ■■■■■■■■ purpose/advantages, ■■■■■■■■ output-■ ■■■■■■/how check ■■■■■■■■ ■■■■■■■■ practical-■ ■■■■■■■■■■■■.

1. Project Overview

[illegible]

2. Technologies / Concepts Used

Concept	Project-■■■ ■■■■■ ■■■■■■■■■■■■	Main Advantage
Spring Boot + REST	REST controllers ■■■■■■■■ APIs	Standard client/API communication
MySQL + JPA/Hibernate	Database + ORM	CRUD ■■■■■■■■ relationships easy
DTO	Request/response objects	Entity/API separation
Mapper	DTO ↔ Entity conversion	Clean service code
Service Layer	Business logic	Maintainable/testable code
Repository	Database operations	Less SQL boilerplate
Validation	@Valid + Jakarta validation	Invalid input early reject
Exception Handling	Custom + global handler	Consistent error responses
Transactions	@Transactional/readOnly	Data consistency
Pagination/Sorting	Pageable based responses	Large data handling
Search/Filtering	Criteria based queries	Find required records
JPA Relationships	Entity relationships	Real-world data modelling
Query Optimization	Lazy loading + optimized queries	Reduce unnecessary DB work
JWT	Authentication token	Stateless authentication
Role Security	ADMIN/TEACHER/STUDENT rules	Controlled access
BCrypt + Password Policy	Password hashing/strength	Better password security
AOP	Service execution logging	Centralized cross-cutting logging
Auditing	Created/Modified timestamps	Record tracking
Scheduler	Scheduled exam processing	Automatic background work
File Upload/Download	/api/files	Document management
Swagger/OpenAPI	Interactive API documentation	Easy manual API testing
CORS	Allowed frontend origins/methods	Frontend integration
Actuator	health/info/metrics	Monitoring
Profiles	dev/test/prod configs	Environment separation
Environment Variables	DB/JWT/password values externalized	Safer configuration
Flyway	DB migration/versioning	Controlled schema changes
JUnit + Mockito	Unit tests	Business logic verification
Integration Testing	Spring/MockMvc/security tests	Layer verification

3. Main Modules

- Student Management
- Teacher Management
- Course Management
- Enrollment Management
- Exam Management
- Grade Management
- Authentication
- File Upload / Download

4. Project Architecture

Client → Controller → DTO + Validation → Service → Mapper → Repository → JPA/Hibernate → MySQL

Security request flow: Client → JWT Filter → Authentication → URL/Method Role Check → Controller → Service. AOP logging service execution- capture `SecurityContextHolder.getContext().getAuthentication()`; Auditing entity create/update information track `AuditingEntityListener`.

5. Project Structure

```
src/main/java/com/example
├── controller
├── service
├── repository
├── entity
├── dto
├── mapper
├── security
├── exception
├── aspect
├── scheduler
├── config

src/main/resources
├── application.yml
├── application-dev.yml
├── application-test.yml
├── application-prod.yml
├── db/migration
```

6. API Testing Order

1. Start MySQL
2. Start Spring Boot in STS
3. Open Swagger
4. Login and get JWT
5. Authorize with Bearer token
6. Student CRUD
7. Teacher CRUD
8. Course CRUD
9. Enrollment CRUD
10. Exam CRUD
11. Grade CRUD
12. Search + Filtering
13. Pagination + Sorting
14. File upload/download
15. Security 401/403 checks
16. Actuator checks
17. MySQL verification
18. Maven tests

7. Swagger Output — How to Check

Swagger URL: http://localhost:8082/swagger-ui/index.html

Login: POST /api/auth/login. Current configuration has configurable default credentials; the sample admin password shown in configuration is Admin@12345. After login, copy the JWT and use Swagger Authorize.

CRUD: GET → HTTP 200; successful create → HTTP 201 where configured; update/delete → success response according to controller.

8. Security Output

No token → **401 Unauthorized**.
Authenticated user without required role → **403 Access Denied**.
Correct role → protected API response.

9. Role Concept

Role	Current URL-level protected access
ADMIN	Students, Teachers, Courses, Enrollments, Exams, Grades, Files
TEACHER	Students, Teachers, Courses, Enrollments, Exams, Grades, Files
STUDENT	Students, Enrollments, Grades, Files

10. Output ■■■■■■ ■■■■■■■■■■■■?

Feature	Where to check	Expected
Startup	STS Console	Application started on port 8082
CRUD	Swagger	JSON + HTTP status
Database	MySQL Workbench	Rows inserted/updated/deleted
Validation	Swagger	Validation error
Exception	Swagger	Standard error response
JWT/Security	Swagger	Token / 401 / 403
AOP	STS Console	Started/Completed method logs
Auditing	API/DB	Created/modified timestamps
Scheduler	STS Console	Scheduled execution when triggered
File	uploads folder + Swagger	File exists + download works
Actuator	Browser	health/info/metrics
Flyway	MySQL	Migration history
Tests	Maven/STS Console	Failures 0, Errors 0, BUILD SUCCESS

11. MySQL Verification

```
USE education_management_db;
SELECT * FROM students;
SELECT * FROM teachers;
SELECT * FROM courses;
SELECT * FROM enrollments;
SELECT * FROM exams;
SELECT * FROM grades;
SELECT * FROM users;
SELECT * FROM flyway_schema_history;
```

12. Actuator Verification

Browser-■■■:
http://localhost:8082/actuator/health
http://localhost:8082/actuator/info
http://localhost:8082/actuator/metrics

13. AOP Logging Verification

Service API call ■■■■■■■■■■■■ STS Console-■■ ■■■■■■■■■■ log ■■■■■■■■■■■■:
Started method: getAllStudents
Completed method: getAllStudents in ... ms

14. Scheduler Verification

ExamScheduler @Scheduled ■■■■■■ upcoming exams process ■■■■■■■■■■. Scheduled execution trigger ■■■
■■■■■■■■■■ STS Console logs ■■■■■■■■■■ application behavior ■■■■■■ verify ■■■■■■■■■■.

15. File Upload / Download Verification

Swagger → /api/files → upload → Execute → success response → project root-■■ uploads/ folder-■■ file ■■■■■■■■■■■■
check → download endpoint ■■■■■■ retrieve check. Current implementation uploaded path-■ normalize/validate ■■■■■■■■■■
unsafe path traversal-■ prevent ■■■■■■■■■■.

16. Testing Verification

Run:
mvn clean test

Current verified project result: **83 tests, 0 failures, 0 errors, 0 skipped, BUILD SUCCESS**. Security integration tests include unauthenticated 401 and insufficient-role 403 checks.

17. Trainer Demo Flow

1. Project objective → 2. Architecture → 3. Swagger login + JWT → 4. Student CRUD → 5. Teacher/Course relationship → 6. Enrollment → 7. Exam/Grade → 8. Search/filter/pagination/sorting → 9. Validation/error → 10. 401/403 security → 11. File upload/download → 12. AOP console log → 13. Auditing → 14. Actuator → 15. Flyway → 16. Maven test result

18. Final Advantages

- Layered maintainable architecture
- DTO + Mapper separation
- Real relational database modelling
- Validation + consistent error handling
- JWT + role-based security + BCrypt
- Transactions + query optimization
- Pagination/search/filtering
- AOP logging + auditing
- Scheduler + file management
- Swagger API documentation/testing
- Actuator monitoring

- Profiles + environment variables
- Flyway migrations
- Unit + integration testing

19. Final Checklist

■ Startup ■ Swagger ■ JWT ■ Roles ■ All CRUD modules ■ Validation ■ Exceptions ■ DTO/Mapper ■ Transactions ■ JPA relationships ■ Pagination/Sorting ■ Search/Filtering ■ Query optimization ■ AOP ■ Auditing ■ Scheduler ■ File upload/download ■ CORS ■ Actuator ■ Profiles ■ Environment variables ■ Flyway ■ JUnit/Mockito ■ Integration tests ■ BUILD SUCCESS ■ MySQL verification

Important: ■■■■ PDF current project source ■■■■■■■■ verified test information-■ base ■■■■■■■■■■. Scheduler ■■■■■■ runtime-only output actual execution trigger ■■■■■■■■ verify ■■■■■■ ■■■■■■■■■■.